

EXPERIMENT-3

AIM:

Create an Employee–Department–Project schema. Insert at least 30 employees across 5 departments and 8 projects. Write SQL queries demonstrating selection, projection, aggregates, GROUP BY, HAVING, CASE expressions, and ORDER BY.

QUERY:

1. Create Database and Tables

```
CREATE DATABASE CompanyDB;
USE CompanyDB;

-- Department Table
CREATE TABLE Department (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(50) NOT NULL,
    Location VARCHAR(50)
);

-- Project Table
CREATE TABLE Project (
    Project_ID INT PRIMARY KEY,
    Project_Name VARCHAR(100) NOT NULL,
    Budget DECIMAL(12,2),
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);

-- Employee Table
CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    Emp_Name VARCHAR(100) NOT NULL,
    Salary DECIMAL(10,2),
    Job_Title VARCHAR(50),
    Dept_ID INT,
    Project_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID),
    FOREIGN KEY (Project_ID) REFERENCES Project(Project_ID)
);
```

2. Insert 5 Departments

```
INSERT INTO Department VALUES
(1, 'IT', 'Delhi'),
(2, 'HR', 'Mumbai'),
(3, 'Finance', 'Bangalore'),
(4, 'Marketing', 'Pune'),
(5, 'Operations', 'Chennai');
```

3. Insert 8 Projects

```
INSERT INTO Project VALUES
(101, 'Website Development', 500000, 1),
(102, 'Mobile Application', 750000, 1),
(103, 'Recruitment System', 300000, 2),
```

```
(104, 'Financial Analysis', 600000, 3),
(105, 'Marketing Campaign', 450000, 4),
(106, 'Customer Management', 550000, 4),
(107, 'Inventory System', 400000, 5),
(108, 'Business Automation', 800000, 5);
```

4. Insert 30 Employees

INSERT INTO Employee VALUES

```
(1, 'Aarav Sharma', 55000, 'Developer', 1, 101),
(2, 'Riya Gupta', 62000, 'Developer', 1, 102),
(3, 'Arjun Verma', 48000, 'Tester', 1, 101),
(4, 'Ananya Singh', 70000, 'Team Lead', 1, 102),
(5, 'Karan Mehta', 45000, 'Support Engineer', 1, 101),
(6, 'Priya Sharma', 58000, 'Developer', 1, 102),
(7, 'Neha Kapoor', 50000, 'HR Executive', 2, 103),
(8, 'Rahul Jain', 65000, 'HR Manager', 2, 103),
(9, 'Simran Kaur', 42000, 'Recruiter', 2, 103),
(10, 'Aditya Roy', 46000, 'HR Executive', 2, 103),

(11, 'Megha Joshi', 55000, 'Recruiter', 2, 103),
(12, 'Vivek Malhotra', 72000, 'HR Manager', 2, 103),
(13, 'Ishita Rao', 60000, 'Accountant', 3, 104),
(14, 'Manish Kumar', 75000, 'Financial Analyst', 3, 104),
(15, 'Pooja Agarwal', 52000, 'Accountant', 3, 104),
(16, 'Rohit Bansal', 68000, 'Financial Analyst', 3, 104),
(17, 'Kavya Nair', 58000, 'Accountant', 3, 104),
(18, 'Sahil Gupta', 80000, 'Finance Manager', 3, 104),
(19, 'Tanya Singh', 47000, 'Marketing Executive', 4, 105),
(20, 'Yash Verma', 56000, 'Marketing Analyst', 4, 105),

(21, 'Nisha Patel', 62000, 'Marketing Manager', 4, 106),
(22, 'Aman Khan', 44000, 'Sales Executive', 4, 106),
(23, 'Shreya Das', 51000, 'Marketing Executive', 4, 105),
(24, 'Dev Sharma', 59000, 'Sales Executive', 4, 106),
(25, 'Akash Yadav', 49000, 'Operations Executive', 5, 107),
(26, 'Sneha Roy', 57000, 'Operations Analyst', 5, 107),
(27, 'Varun Singh', 63000, 'Operations Manager', 5, 108),
(28, 'Komal Gupta', 46000, 'Operations Executive', 5, 107),
(29, 'Nitin Sharma', 54000, 'Process Analyst', 5, 108),
(30, 'Aditi Mehta', 69000, 'Operations Manager', 5, 108);
```

SQL Queries

5. Selection

Selection retrieves rows that satisfy a condition.

Employees having salary greater than ₹60,000

SELECT *

FROM Employee

WHERE Salary > 60000;

Emp_ID	Emp_Name	Salary	Job_Title	Dept_ID	Project_ID
2	Riya Gupta	62000.00	Developer	1	102
4	Ananya Singh	70000.00	Team Lead	1	102
8	Rahul Jain	65000.00	HR Manager	2	103
12	Vivek Malhotra	72000.00	HR Manager	2	103
14	Manish Kumar	75000.00	Financial Analyst	3	104
16	Rohit Bansal	68000.00	Financial Analyst	3	104
18	Sahil Gupta	80000.00	Finance Manager	3	104
21	Nisha Patel	62000.00	Marketing Manager	4	106
27	Varun Singh	63000.00	Operations Manager	5	108
30	Aditi Mehta	69000.00	Operations Manager	5	108

6. Projection

Projection retrieves only specific columns from a table.

SELECT Emp_ID, Emp_Name, Salary
FROM Employee;

Emp_ID	Emp_Name	Salary
1	Aarav Sharma	55000.00
2	Riya Gupta	62000.00
3	Arjun Verma	48000.00
4	Ananya Singh	70000.00
5	Karan Mehta	45000.00
6	Priya Sharma	58000.00
7	Neha Kapoor	50000.00
8	Rahul Jain	65000.00
9	Simran Kaur	42000.00
10	Aditya Roy	46000.00
11	Megha Joshi	55000.00
12	Vivek Malhotra	72000.00
13	Ishita Rao	60000.00
14	Manish Kumar	75000.00
15	Pooja Agarwal	52000.00
16	Rohit Bansal	68000.00
17	Kavya Nair	58000.00
18	Sahil Gupta	80000.00
19	Tanya Singh	47000.00
20	Yash Verma	56000.00
21	Nisha Patel	62000.00
22	Aman Khan	44000.00
23	Shreya Das	51000.00
24	Dev Sharma	59000.00
25	Akash Yadav	49000.00
26	Sneha Roy	57000.00
27	Varun Singh	63000.00
28	Komal Gupta	46000.00
29	Nitin Sharma	54000.00
30	Aditi Mehta	69000.00

7. Aggregate Functions

COUNT, SUM, AVG, MAX, MIN

SELECT

COUNT(*) AS Total_Employees,

SUM(Salary) AS Total_Salary,

AVG(Salary) AS Average_Salary,

MAX(Salary) AS Highest_Salary,

MIN(Salary) AS Lowest_Salary

FROM Employee;

Total_Employees	Total_Salary	Average_Salary	Highest_Salary	Lowest_Salary
30	1718000.00	57266.666667	80000.00	42000.00

8. GROUP BY

Find the average salary of each department:

SELECT Dept_ID, AVG(Salary) AS Average_Salary

FROM Employee

GROUP BY Dept_ID;

Dept_ID	Average_Salary
1	56333.333333
2	55000.000000
3	65500.000000
4	53166.666667
5	56333.333333

Using the department name:

SELECT d.Dept_Name, COUNT(e.Emp_ID) AS Employee_Count

FROM Department d

JOIN Employee e

ON d.Dept_ID = e.Dept_ID

GROUP BY d.Dept_Name;

Dept_Name	Employee_Count
IT	6
HR	6
Finance	6
Marketing	6
Operations	6

9. HAVING

HAVING is used to filter groups created by GROUP BY.

Departments having more than 5 employees

SELECT Dept_ID, COUNT(*) AS Employee_Count

FROM Employee

GROUP BY Dept_ID

HAVING COUNT(*) > 5;

Dept_ID	Employee_Count
1	6
2	6
3	6
4	6
5	6

10. CASE Expression

Classify employees according to their salary:

SELECT

Emp_Name, Salary,

CASE

WHEN Salary >= 70000 THEN 'High Salary'

WHEN Salary >= 50000 THEN 'Medium Salary'

ELSE 'Low Salary'

END AS Salary_Category

FROM Employee;

Emp_Name	Salary	Salary_Category
Aarav Sharma	55000.00	Medium Salary
Riya Gupta	62000.00	Medium Salary
Arjun Verma	48000.00	Low Salary
Ananya Singh	70000.00	High Salary
Karan Mehta	45000.00	Low Salary
Priya Sharma	58000.00	Medium Salary
Neha Kapoor	50000.00	Medium Salary
Rahul Jain	65000.00	Medium Salary
Simran Kaur	42000.00	Low Salary
Aditya Roy	46000.00	Low Salary
Megha Joshi	55000.00	Medium Salary
Vivek Malhotra	72000.00	High Salary
Ishita Rao	60000.00	Medium Salary
Manish Kumar	75000.00	High Salary
Pooja Agarwal	52000.00	Medium Salary
Rohit Bansal	68000.00	Medium Salary
Kavya Nair	58000.00	Medium Salary
Sahil Gupta	80000.00	High Salary
Tanya Singh	47000.00	Low Salary
Yash Verma	56000.00	Medium Salary
Nisha Patel	62000.00	Medium Salary
Aman Khan	44000.00	Low Salary
Shreya Das	51000.00	Medium Salary
Dev Sharma	59000.00	Medium Salary
Akash Yadav	49000.00	Low Salary
Sneha Roy	57000.00	Medium Salary
Varun Singh	63000.00	Medium Salary
Komal Gupta	46000.00	Low Salary
Nitin Sharma	54000.00	Medium Salary
Aditi Mehta	69000.00	Medium Salary

11. ORDER BY**Sort employees by salary in ascending order****SELECT Emp_Name, Salary****FROM Employee****ORDER BY Salary ASC;**

Emp_Name	Salary
Simran Kaur	42000.00
Aman Khan	44000.00
Karan Mehta	45000.00
Aditya Roy	46000.00
Komal Gupta	46000.00
Tanya Singh	47000.00
Arjun Verma	48000.00
Akash Yadav	49000.00
Neha Kapoor	50000.00
Shreya Das	51000.00
Pooja Agarwal	52000.00
Nitin Sharma	54000.00
Megha Joshi	55000.00
Aarav Sharma	55000.00
Yash Verma	56000.00
Sneha Roy	57000.00
Kavya Nair	58000.00
Priya Sharma	58000.00
Dev Sharma	59000.00
Ishita Rao	60000.00
Nisha Patel	62000.00
Riya Gupta	62000.00
Varun Singh	63000.00
Rahul Jain	65000.00
Rohit Bansal	68000.00
Aditi Mehta	69000.00
Ananya Singh	70000.00
Vivek Malhotra	72000.00
Manish Kumar	75000.00
Sahil Gupta	80000.00

EXPERIMENT 4

AIM:

Using the Employee schema, write queries with INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare execution plans using EXPLAIN.

SQL Queries

1. INNER JOIN

Display employees along with their department names.

SELECT

**e.Emp_ID,
e.Emp_Name,
e.Salary,
d.Dept_Name**

FROM Employee e

INNER JOIN Department d

ON e.Dept_ID = d.Dept_ID;

Emp_ID	Emp_Name	Salary	Dept_Name
1	Aarav Sharma	55000.00	IT
2	Riya Gupta	62000.00	IT
3	Arjun Verma	48000.00	IT
4	Ananya Singh	70000.00	IT
5	Karan Mehta	45000.00	IT
6	Priya Sharma	58000.00	IT
7	Neha Kapoor	50000.00	HR
8	Rahul Jain	65000.00	HR
9	Simran Kaur	42000.00	HR
10	Aditya Roy	46000.00	HR
11	Megha Joshi	55000.00	HR
12	Vivek Malhotra	72000.00	HR
13	Ishita Rao	60000.00	Finance
14	Manish Kumar	75000.00	Finance
15	Pooja Agarwal	52000.00	Finance
16	Rohit Bansal	68000.00	Finance
17	Kavya Nair	58000.00	Finance
18	Sahil Gupta	80000.00	Finance
19	Tanya Singh	47000.00	Marketing
20	Yash Verma	56000.00	Marketing
21	Nisha Patel	62000.00	Marketing
22	Aman Khan	44000.00	Marketing
23	Shreya Das	51000.00	Marketing
24	Dev Sharma	59000.00	Marketing
25	Akash Yadav	49000.00	Operations
26	Sneha Roy	57000.00	Operations
27	Varun Singh	63000.00	Operations
28	Komal Gupta	46000.00	Operations
29	Nitin Sharma	54000.00	Operations
30	Aditi Mehta	69000.00	Operations

2. LEFT JOIN

Display all departments and their employees.

SELECT

d.Dept_ID, d.Dept_Name, e.Emp_Name

FROM Department d

LEFT JOIN Employee e

ON d.Dept_ID = e.Dept_ID;

Dept_ID	Dept_Name	Emp_Name
1	IT	Aarav Sharma
1	IT	Riya Gupta
1	IT	Arjun Verma
1	IT	Ananya Singh
1	IT	Karan Mehta
1	IT	Priya Sharma
2	HR	Neha Kapoor
2	HR	Rahul Jain
2	HR	Simran Kaur
2	HR	Aditya Roy
2	HR	Megha Joshi
2	HR	Vivek Malhotra
3	Finance	Ishita Rao
3	Finance	Manish Kumar
3	Finance	Pooja Agarwal
3	Finance	Rohit Bansal
3	Finance	Kavya Nair
3	Finance	Sahil Gupta
4	Marketing	Tanya Singh
4	Marketing	Yash Verma
4	Marketing	Nisha Patel
4	Marketing	Aman Khan
4	Marketing	Shreya Das
4	Marketing	Dev Sharma
5	Operations	Akash Yadav
5	Operations	Sneha Roy
5	Operations	Varun Singh
5	Operations	Komal Gupta
5	Operations	Nitin Sharma
5	Operations	Aditi Mehta

3. SELF JOIN

Find pairs of employees who work in the **same department**.

SELECT

e1.Emp_Name AS Employee1,

e2.Emp_Name AS Employee2,

e1.Dept_ID

FROM Employee e1

JOIN Employee e2

ON e1.Dept_ID = e2.Dept_ID

AND e1.Emp_ID < e2.Emp_ID;

Employee1	Employee2	Dept_ID
Aarav Sharma	Riya Gupta	1
Aarav Sharma	Arjun Verma	1
Aarav Sharma	Ananya Singh	1
Aarav Sharma	Karan Mehta	1
Aarav Sharma	Priya Sharma	1
Riya Gupta	Arjun Verma	1
Riya Gupta	Ananya Singh	1
Riya Gupta	Karan Mehta	1
Riya Gupta	Priya Sharma	1
Arjun Verma	Ananya Singh	1
Arjun Verma	Karan Mehta	1
Arjun Verma	Priya Sharma	1
Ananya Singh	Karan Mehta	1
Ananya Singh	Priya Sharma	1
Karan Mehta	Priya Sharma	1
Neha Kapoor	Rahul Jain	2
Neha Kapoor	Simran Kaur	2
Neha Kapoor	Aditya Roy	2
Neha Kapoor	Megha Joshi	2
Neha Kapoor	Vivek Malhotra	2
Rahul Jain	Simran Kaur	2
Rahul Jain	Aditya Roy	2
Rahul Jain	Megha Joshi	2
Rahul Jain	Vivek Malhotra	2
Simran Kaur	Aditya Roy	2
Simran Kaur	Megha Joshi	2
Simran Kaur	Vivek Malhotra	2
Aditya Roy	Megha Joshi	2
Aditya Roy	Vivek Malhotra	2
Megha Joshi	Vivek Malhotra	2
Ishita Rao	Manish Kumar	3
Ishita Rao	Pooja Agarwal	3
Ishita Rao	Rohit Bansal	3
Ishita Rao	Kavya Nair	3
Ishita Rao	Sahil Gupta	3
Manish Kumar	Pooja Agarwal	3
Manish Kumar	Rohit Bansal	3
Manish Kumar	Kavya Nair	3
Manish Kumar	Sahil Gupta	3
Pooja Agarwal	Rohit Bansal	3
Pooja Agarwal	Kavya Nair	3
Pooja Agarwal	Sahil Gupta	3
Rohit Bansal	Kavya Nair	3
Rohit Bansal	Sahil Gupta	3

Kavya Nair	Sahil Gupta	3
Tanya Singh	Yash Verma	4
Tanya Singh	Nisha Patel	4
Tanya Singh	Aman Khan	4
Tanya Singh	Shreya Das	4
Tanya Singh	Dev Sharma	4
Yash Verma	Nisha Patel	4
Yash Verma	Aman Khan	4
Yash Verma	Shreya Das	4
Yash Verma	Dev Sharma	4
Nisha Patel	Aman Khan	4
Nisha Patel	Shreya Das	4
Nisha Patel	Dev Sharma	4
Aman Khan	Shreya Das	4
Aman Khan	Dev Sharma	4
Shreya Das	Dev Sharma	4
Akash Yadav	Sneha Roy	5
Akash Yadav	Varun Singh	5
Akash Yadav	Komal Gupta	5
Akash Yadav	Nitin Sharma	5
Akash Yadav	Aditi Mehta	5
Sneha Roy	Varun Singh	5
Sneha Roy	Komal Gupta	5
Sneha Roy	Nitin Sharma	5
Sneha Roy	Aditi Mehta	5
Varun Singh	Komal Gupta	5
Varun Singh	Nitin Sharma	5
Varun Singh	Aditi Mehta	5
Komal Gupta	Nitin Sharma	5
Komal Gupta	Aditi Mehta	5
Nitin Sharma	Aditi Mehta	5

4. Three-Way JOIN

Display employee name, department name, and project name.

SELECT

e.Emp_Name, d.Dept_Name, p.Project_Name, e.Salary

FROM Employee e

JOIN Department d

ON e.Dept_ID = d.Dept_ID

JOIN Project p

ON e.Project_ID = p.Project_ID;

Emp_Name	Dept_Name	Project_Name	Salary
Aarav Sharma	IT	Website Development	55000.00
Riya Gupta	IT	Mobile Application	62000.00
Arjun Verma	IT	Website Development	48000.00
Ananya Singh	IT	Mobile Application	70000.00
Karan Mehta	IT	Website Development	45000.00
Priya Sharma	IT	Mobile Application	58000.00
Neha Kapoor	HR	Recruitment System	50000.00
Rahul Jain	HR	Recruitment System	65000.00
Simran Kaur	HR	Recruitment System	42000.00
Aditya Roy	HR	Recruitment System	46000.00
Megha Joshi	HR	Recruitment System	55000.00
Vivek Malhotra	HR	Recruitment System	72000.00
Ishita Rao	Finance	Financial Analysis	60000.00
Manish Kumar	Finance	Financial Analysis	75000.00

Pooja Agarwal	Finance	Financial Analysis	52000.00
Rohit Bansal	Finance	Financial Analysis	68000.00
Kavya Nair	Finance	Financial Analysis	58000.00
Sahil Gupta	Finance	Financial Analysis	80000.00
Tanya Singh	Marketing	Marketing Campaign	47000.00
Yash Verma	Marketing	Marketing Campaign	56000.00
Nisha Patel	Marketing	Customer Management	62000.00
Aman Khan	Marketing	Customer Management	44000.00
Shreya Das	Marketing	Marketing Campaign	51000.00
Dev Sharma	Marketing	Customer Management	59000.00
Akash Yadav	Operations	Inventory System	49000.00
Sneha Roy	Operations	Inventory System	57000.00
Varun Singh	Operations	Business Automation	63000.00
Komal Gupta	Operations	Inventory System	46000.00
Nitin Sharma	Operations	Business Automation	54000.00
Aditi Mehta	Operations	Business Automation	69000.00

5. Correlated Subquery

Find employees whose salary is greater than the **average salary of their own department**.

SELECT

e.Emp_ID, e.Emp_Name, e.Dept_ID, e.Salary

FROM Employee e

WHERE e.Salary >

(

SELECT AVG(e2.Salary)

FROM Employee e2

WHERE e2.Dept_ID = e.Dept_ID

);

Emp_ID	Emp_Name	Dept_ID	Salary
2	Riya Gupta	1	62000.00
4	Ananya Singh	1	70000.00
6	Priya Sharma	1	58000.00
8	Rahul Jain	2	65000.00
12	Vivek Malhotra	2	72000.00
14	Manish Kumar	3	75000.00
16	Rohit Bansal	3	68000.00
18	Sahil Gupta	3	80000.00
20	Yash Verma	4	56000.00
21	Nisha Patel	4	62000.00
24	Dev Sharma	4	59000.00
26	Sneha Roy	5	57000.00
27	Varun Singh	5	63000.00
30	Aditi Mehta	5	69000.00

6. EXISTS

Find departments that have at least one employee earning more than ₹70,000.

SELECT

d.Dept_ID,

d.Dept_Name

FROM Department d

WHERE EXISTS

(

SELECT 1

FROM Employee e

WHERE e.Dept_ID = d.Dept_ID

AND e.Salary > 70000

);

Dept_ID	Dept_Name
2	HR
3	Finance

7. EXISTS with Projects

Find employees who are assigned to a project.

SELECT

e.Emp_ID,
e.Emp_Name

FROM Employee e

WHERE EXISTS

(
SELECT 1
FROM Project p
WHERE p.Project_ID = e.Project_ID
);

Emp_ID	Emp_Name
1	Aarav Sharma
3	Arjun Verma
5	Karan Mehta
2	Riya Gupta
4	Ananya Singh
6	Priya Sharma
7	Neha Kapoor
8	Rahul Jain
9	Simran Kaur
10	Aditya Roy
11	Megha Joshi
12	Vivek Malhotra
13	Ishita Rao
14	Manish Kumar
15	Pooja Agarwal
16	Rohit Bansal
17	Kavya Nair
18	Sahil Gupta
19	Tanya Singh
20	Yash Verma
23	Shreya Das
21	Nisha Patel
22	Aman Khan
24	Dev Sharma
25	Akash Yadav
26	Sneha Roy
28	Komal Gupta
27	Varun Singh
29	Nitin Sharma
30	Aditi Mehta

8. Simulated INTERSECT

MySQL does not traditionally use the INTERSECT operator in the same way as some other SQL systems, so we can simulate it using INNER JOIN.

Find employees who belong to the IT department AND have salary greater than ₹60,000

```
SELECT Emp_ID, Emp_Name
```

```
FROM Employee
```

```
WHERE Dept_ID = 1
```

```
AND Emp_ID IN
```

```
(
  SELECT Emp_ID
  FROM Employee
  WHERE Salary > 60000
);
```

Emp_ID	Emp_Name
2	Riya Gupta
4	Ananya Singh

9. Simulated EXCEPT

Suppose we want employees who are **not working in the IT department**.

```
SELECT Emp_ID, Emp_Name
```

```
FROM Employee
```

```
WHERE Dept_ID <> 1;
```

Emp_ID	Emp_Name
7	Neha Kapoor
8	Rahul Jain
9	Simran Kaur
10	Aditya Roy
11	Megha Joshi
12	Vivek Malhotra
13	Ishita Rao
14	Manish Kumar
15	Pooja Agarwal
16	Rohit Bansal
17	Kavya Nair
18	Sahil Gupta
19	Tanya Singh
20	Yash Verma
21	Nisha Patel
22	Aman Khan
23	Shreya Das
24	Dev Sharma
25	Akash Yadav
26	Sneha Roy
27	Varun Singh
28	Komal Gupta
29	Nitin Sharma
30	Aditi Mehta

10. EXPLAIN – INNER JOIN

EXPLAIN shows how MySQL plans to execute a query.

EXPLAIN

SELECT

**e.Emp_Name,
d.Dept_Name**

FROM Employee e

INNER JOIN Department d

ON e.Dept_ID = d.Dept_ID;

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	d	null	ALL	PRIMARY	null	null	null	5	100	null
1	SIMPLE	e	null	ref	Dept_ID	Dept_ID	5	db_4546kyfyj.d.Dept_ID	6	100	null

11. EXPLAIN – LEFT JOIN

EXPLAIN

SELECT

**d.Dept_Name,
e.Emp_Name**

FROM Department d

LEFT JOIN Employee e

ON d.Dept_ID = e.Dept_ID;

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	d	null	ALL	null	null	null	null	5	100	null
1	SIMPLE	e	null	ref	Dept_ID	Dept_ID	5	db_4546kyfyj.d.Dept_ID	6	100	null

12. EXPLAIN – Three-Way JOIN

EXPLAIN

SELECT

**e.Emp_Name,
d.Dept_Name,
p.Project_Name**

FROM Employee e

JOIN Department d

ON e.Dept_ID = d.Dept_ID

JOIN Project p

ON e.Project_ID = p.Project_ID;

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	d	null	ALL	PRIMARY	null	null	null	5	100	null
1	SIMPLE	e	null	ref	Dept_ID,Project_ID	Dept_ID	5	db_4546kyfyj.d.Dept_ID	6	100	Using where
1	SIMPLE	p	null	eq_ref	PRIMARY	PRIMARY	4	db_4546kyfyj.e.Project_ID	1	100	null

13. EXPLAIN – Correlated Subquery

EXPLAIN

SELECT

**e.Emp_Name,
e.Salary**

FROM Employee e

WHERE e.Salary >

(

SELECT AVG(e2.Salary)

FROM Employee e2

WHERE e2.Dept_ID = e.Dept_ID

);

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	e	null	ALL	null	null	null	null	30	100	Using where
2	DEPENDENT SUBQUERY	e2	null	ref	Dept_ID	Dept_ID	5	db_4546kyfyj.e.Dept_ID	6	100	null

EXPERIMENT 5

AIM:

Create SQL views for department salary summary and employee hierarchy. Test updatability of views. Implement a recursive CTE to display reporting chains.

1. Add Manager Information to Employee Table

Our original Employee table does not have a reporting/manager relationship. Add a Manager_ID column.

ALTER TABLE Employee

ADD Manager_ID INT NULL;

ALTER TABLE Employee

ADD CONSTRAINT fk_manager

FOREIGN KEY (Manager_ID) REFERENCES Employee(Emp_ID);

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0	Records: 0 Duplicates: 0 Warnings: 0	10	0	0
0	30	0	Records: 30 Duplicates: 0 Warnings: 0	2	0	0

Now assign managers. The top-level managers have NULL.

UPDATE Employee

SET Manager_ID = CASE

WHEN Emp_ID IN (1, 2, 3, 4, 5, 6) THEN NULL

WHEN Emp_ID IN (7, 8, 9, 10, 11, 12) THEN 8

WHEN Emp_ID IN (13, 14, 15, 16, 17, 18) THEN 18

WHEN Emp_ID IN (19, 20, 21, 22, 23, 24) THEN 21

WHEN Emp_ID IN (25, 26, 27, 28, 29, 30) THEN 27

END;

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	30	0	Rows matched: 30 Changed: 24 Warnings: 0	34	0	24

You can check the hierarchy using:

SELECT Emp_ID, Emp_Name, Manager_ID

FROM Employee

ORDER BY Emp_ID;

Emp_ID	Emp_Name	Manager_ID
1	Aarav Sharma	null
2	Riya Gupta	null
3	Arjun Verma	null
4	Ananya Singh	null
5	Karan Mehta	null
6	Priya Sharma	null
7	Neha Kapoor	8
8	Rahul Jain	8
9	Simran Kaur	8
10	Aditya Roy	8
11	Megha Joshi	8
12	Vivek Malhotra	8
13	Ishita Rao	18
14	Manish Kumar	18

15	Pooja Agarwal	18
16	Rohit Bansal	18
17	Kavya Nair	18
18	Sahil Gupta	18
19	Tanya Singh	21
20	Yash Verma	21
21	Nisha Patel	21
22	Aman Khan	21
23	Shreya Das	21
24	Dev Sharma	21
25	Akash Yadav	27
26	Sneha Roy	27
27	Varun Singh	27
28	Komal Gupta	27
29	Nitin Sharma	27
30	Aditi Mehta	27

2. View for Department Salary Summary

Create a view showing the number of employees, total salary, average salary, minimum salary, and maximum salary for every department.

CREATE VIEW Department_Salary_Summary AS

SELECT

**d.Dept_ID, d.Dept_Name,
COUNT(e.Emp_ID) AS Total_Employees,
SUM(e.Salary) AS Total_Salary,
AVG(e.Salary) AS Average_Salary,
MIN(e.Salary) AS Minimum_Salary,
MAX(e.Salary) AS Maximum_Salary**

FROM Department d

LEFT JOIN Employee e

ON d.Dept_ID = e.Dept_ID

GROUP BY d.Dept_ID, d.Dept_Name;

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

Display the view

SELECT *

FROM Department_Salary_Summary;

Dept_ID	Dept_Name	Total_Employees	Total_Salary	Average_Salary	Minimum_Salary	Maximum_Salary
1	IT	6	338000.00	56333.333333	45000.00	70000.00
2	HR	6	330000.00	55000.000000	42000.00	72000.00
3	Finance	6	393000.00	65500.000000	52000.00	80000.00
4	Marketing	6	319000.00	53166.666667	44000.00	62000.00
5	Operations	6	338000.00	56333.333333	46000.00	69000.00

3. Employee Hierarchy View

Create a view showing each employee and their immediate manager.

CREATE VIEW Employee_Hierarchy AS

SELECT

**e.Emp_ID,
e.Emp_Name AS Employee_Name,
e.Job_Title, e.Dept_ID, e.Manager_ID,
m.Emp_Name AS Manager_Name**

FROM Employee e

LEFT JOIN Employee m

ON e.Manager_ID = m.Emp_ID;

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

Display the hierarchy

```
SELECT *
FROM Employee_Hierarchy;
```

Emp_ID	Employee_Name	Job_Title	Dept_ID	Manager_ID	Manager_Name
1	Aarav Sharma	Developer	1	null	null
2	Riya Gupta	Developer	1	null	null
3	Arjun Verma	Tester	1	null	null
4	Ananya Singh	Team Lead	1	null	null
5	Karan Mehta	Support Engineer	1	null	null
6	Priya Sharma	Developer	1	null	null
7	Neha Kapoor	HR Executive	2	8	Rahul Jain
8	Rahul Jain	HR Manager	2	8	Rahul Jain
9	Simran Kaur	Recruiter	2	8	Rahul Jain
10	Aditya Roy	HR Executive	2	8	Rahul Jain
11	Megha Joshi	Recruiter	2	8	Rahul Jain
12	Vivek Malhotra	HR Manager	2	8	Rahul Jain
13	Ishita Rao	Accountant	3	18	Sahil Gupta
14	Manish Kumar	Financial Analyst	3	18	Sahil Gupta
15	Pooja Agarwal	Accountant	3	18	Sahil Gupta
16	Rohit Bansal	Financial Analyst	3	18	Sahil Gupta
17	Kavya Nair	Accountant	3	18	Sahil Gupta
18	Sahil Gupta	Finance Manager	3	18	Sahil Gupta
19	Tanya Singh	Marketing Executive	4	21	Nisha Patel
20	Yash Verma	Marketing Analyst	4	21	Nisha Patel
21	Nisha Patel	Marketing Manager	4	21	Nisha Patel
22	Aman Khan	Sales Executive	4	21	Nisha Patel
23	Shreya Das	Marketing Executive	4	21	Nisha Patel
24	Dev Sharma	Sales Executive	4	21	Nisha Patel
25	Akash Yadav	Operations Executive	5	27	Varun Singh
26	Sneha Roy	Operations Analyst	5	27	Varun Singh
27	Varun Singh	Operations Manager	5	27	Varun Singh
28	Komal Gupta	Operations Executive	5	27	Varun Singh
29	Nitin Sharma	Process Analyst	5	27	Varun Singh
30	Aditi Mehta	Operations Manager	5	27	Varun Singh

4. Test View Updatability

A view is **updatable** when modifications made through the view can be translated unambiguously to the underlying table.

Test 1 — Update Employee Hierarchy View

Try:

```
UPDATE Employee_Hierarchy
SET Employee_Name = 'Aarav Kumar'
WHERE Emp_ID = 1;
```

This view contains a **self-join** between two instances of Employee, so it is not a simple single-table view. MySQL will generally reject modifications to this view because it is not inherently updatable.

Error: The target table Employee_Hierarchy of the UPDATE is not updatable

Therefore, use:

```
SELECT *
FROM Employee_Hierarchy
WHERE Emp_ID = 1;
```

to verify the data instead.

Emp_ID	Employee_Name	Job_Title	Dept_ID	Manager_ID	Manager_Name
1	Aarav Sharma	Developer	1	null	null

5. Test Department Salary Summary View

Try:

UPDATE Department_Salary_Summary

SET Average_Salary = 60000

WHERE Dept_ID = 1;

This view is **not updatable** because it contains:

- GROUP BY
- aggregate functions such as SUM(), AVG(), COUNT(), MIN(), and MAX()
- a LEFT JOIN

The value Average_Salary is calculated from multiple employee rows, so MySQL cannot determine which individual employee record should be changed.

Error: The target table Department_Salary_Summary of the UPDATE is not updatable

6. Create a Simple Updatable View

To demonstrate an actually updatable view, create a simple view containing rows directly from Employee.

CREATE VIEW Employee_Salary_View AS

SELECT

Emp_ID, Emp_Name, Salary, Dept_ID

FROM Employee;

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

View the data:

SELECT *

FROM Employee_Salary_View;

Emp_ID	Emp_Name	Salary	Dept_ID
1	Aarav Sharma	55000.00	1
2	Riya Gupta	62000.00	1
3	Arjun Verma	48000.00	1
4	Ananya Singh	70000.00	1
5	Karan Mehta	45000.00	1
6	Priya Sharma	58000.00	1
7	Neha Kapoor	50000.00	2
8	Rahul Jain	65000.00	2
9	Simran Kaur	42000.00	2
10	Aditya Roy	46000.00	2
11	Megha Joshi	55000.00	2
12	Vivek Malhotra	72000.00	2
13	Ishita Rao	60000.00	3
14	Manish Kumar	75000.00	3
15	Pooja Agarwal	52000.00	3
16	Rohit Bansal	68000.00	3
17	Kavya Nair	58000.00	3
18	Sahil Gupta	80000.00	3
19	Tanya Singh	47000.00	4
20	Yash Verma	56000.00	4
21	Nisha Patel	62000.00	4
22	Aman Khan	44000.00	4
23	Shreya Das	51000.00	4
24	Dev Sharma	59000.00	4
25	Akash Yadav	49000.00	5
26	Sneha Roy	57000.00	5
27	Varun Singh	63000.00	5
28	Komal Gupta	46000.00	5
29	Nitin Sharma	54000.00	5
30	Aditi Mehta	69000.00	5

Now update an employee through the view:

```
UPDATE Employee_Salary_View
SET Salary = 65000
WHERE Emp_ID = 1;
```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	1	0	Rows matched: 1 Changed: 1 Warnings: 0	2	0	1

Check the original table:

```
SELECT Emp_ID, Emp_Name, Salary
FROM Employee
WHERE Emp_ID = 1;
```

Emp_ID	Emp_Name	Salary
1	Aarav Sharma	65000.00

7. Recursive CTE

A **recursive Common Table Expression (CTE)** is useful for hierarchical data such as:

The recursive CTE has two parts:

1. **Anchor query** — finds the starting/top-level employees.
2. **Recursive query** — repeatedly finds employees reporting to the previous level.

8. Display Complete Reporting Chains

WITH RECURSIVE EmployeeChain AS

```
(
  -- Anchor member
  SELECT
    Emp_ID, Emp_Name, Manager_ID, 0 AS Level,
    CAST(Emp_Name AS CHAR(1000)) AS Reporting_Chain
  FROM Employee
  WHERE Manager_ID IS NULL
  UNION ALL
  -- Recursive member
  SELECT
    e.Emp_ID, e.Emp_Name, e.Manager_ID, ec.Level + 1,
    CONCAT(ec.Reporting_Chain, ' -> ', e.Emp_Name)
  FROM Employee e
  INNER JOIN EmployeeChain ec
    ON e.Manager_ID = ec.Emp_ID
)
SELECT
  Emp_ID, Emp_Name, Manager_ID, Level,
  Reporting_Chain
FROM EmployeeChain
ORDER BY Reporting_Chain;
```

Emp_ID	Emp_Name	Manager_ID	Level	Reporting_Chain
1	Aarav Sharma	null	0	Aarav Sharma
4	Ananya Singh	null	0	Ananya Singh
3	Arjun Verma	null	0	Arjun Verma
5	Karan Mehta	null	0	Karan Mehta
6	Priya Sharma	null	0	Priya Sharma
2	Riya Gupta	null	0	Riya Gupta

9. Display Hierarchy Level by Level

If you only want to see the employee's hierarchy level:

WITH RECURSIVE EmployeeHierarchy AS

```
(
  SELECT
    Emp_ID, Emp_Name, Manager_ID, 0 AS Hierarchy_Level
```

FROM Employee
WHERE Manager_ID IS NULL

UNION ALL
SELECT
 e.Emp_ID, e.Emp_Name, e.Manager_ID, eh.Hierarchy_Level + 1
FROM Employee e
JOIN EmployeeHierarchy eh
 ON e.Manager_ID = eh.Emp_ID

)

SELECT
 Emp_ID, Emp_Name, Manager_ID, Hierarchy_Level
FROM EmployeeHierarchy
ORDER BY Hierarchy_Level, Emp_ID;

Emp_ID	Emp_Name	Manager_ID	Hierarchy_Level
1	Aarav Sharma	null	0
2	Riya Gupta	null	0
3	Arjun Verma	null	0
4	Ananya Singh	null	0
5	Karan Mehta	null	0
6	Priya Sharma	null	0

10. Find All Employees Reporting to One Manager

For example, find the complete hierarchy below employee 4:

WITH RECURSIVE Subordinates AS

(

SELECT
 Emp_ID, Emp_Name, Manager_ID, 0 AS Level
FROM Employee
WHERE Emp_ID = 4

UNION ALL
SELECT
 e.Emp_ID, e.Emp_Name, e.Manager_ID, s.Level + 1
FROM Employee e
JOIN Subordinates s
 ON e.Manager_ID = s.Emp_ID

)

SELECT *
FROM Subordinates;

Emp_ID	Emp_Name	Manager_ID	Level
4	Ananya Singh	null	0

EXPERIMENT 6

AIM:

Create a stored procedure transfer_employee(emp_id, new_dept_id) with validation and error handling. Implement triggers for salary validation and audit logging. Test edge cases.

SQL QUERIES

1. Create Audit Table

First, create a table to store salary changes.

```
CREATE TABLE Salary_Audit (
  Audit_ID INT AUTO_INCREMENT PRIMARY KEY,
  Emp_ID INT NOT NULL,
  Old_Salary DECIMAL(10,2),
  New_Salary DECIMAL(10,2),
  Changed_At TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  Action_Type VARCHAR(20)
);
```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

2. Trigger for Salary Validation

Create a BEFORE INSERT trigger:

```
CREATE TRIGGER validate_salary_insert
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
  IF NEW.Salary IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Salary cannot be NULL';

    ELSEIF NEW.Salary <= 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Salary must be greater than zero';

    ELSEIF NEW.Salary < 10000 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Salary cannot be less than 10000';

    ELSEIF NEW.Salary > 1000000 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Salary cannot exceed 1000000';
  END IF;
END
```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

SHOW TRIGGERS;

Trigger	Event	Table	Statement
validate_salary_insert	INSERT	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG
validate_salary_update	UPDATE	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG

9. Salary Audit Trigger

This trigger automatically records every successful salary change.

```
CREATE TRIGGER salary_audit
AFTER UPDATE ON Employee
FOR EACH ROW
```

BEGIN

IF OLD.Salary <> NEW.Salary THEN

INSERT INTO Salary_Audit

(
Emp_ID,
Old_Salary,
New_Salary,
Action_Type
)

VALUES

(
NEW.Emp_ID,
OLD.Salary,
NEW.Salary,
'SALARY UPDATE'
);

END IF;

END

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

SHOW TRIGGERS;

Trigger	Event	Table	Statement
validate_salary_insert	INSERT	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG
validate_salary_update	UPDATE	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG
salary_audit	UPDATE	employee	BEGIN IF OLD.Salary <> NEW.Salary THEN INSERT INTO Salary_Audit (Emp_ID, Old_Salary, New_Salary, Action_Type) VALUES (NEW.Emp_ID, OLI

10. Create Stored Procedure

The procedure transfer_employee() transfers an employee to another department.

CREATE PROCEDURE transfer_employee(

IN p_emp_id INT,
IN p_new_dept_id INT

)

BEGIN

DECLARE v_emp_count INT DEFAULT 0;
DECLARE v_dept_count INT DEFAULT 0;
DECLARE v_current_dept INT DEFAULT 0;

DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

ROLLBACK;
RESIGNAL;

END;

START TRANSACTION;

SELECT COUNT(*)
INTO v_emp_count
FROM Employee
WHERE Emp_ID = p_emp_id;

IF v_emp_count = 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Employee does not exist';
END IF;

SELECT COUNT(*)

```

INTO v_dept_count
FROM Department
WHERE Dept_ID = p_new_dept_id;

```

```

IF v_dept_count = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Department does not exist';
END IF;

```

```

SELECT Dept_ID
INTO v_current_dept
FROM Employee
WHERE Emp_ID = p_emp_id;

```

```

IF v_current_dept = p_new_dept_id THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Employee is already in this department';
END IF;

```

```

UPDATE Employee
SET Dept_ID = p_new_dept_id
WHERE Emp_ID = p_emp_id;

```

```

COMMIT;

```

```

END

```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

11. Check Stored Procedure

```

SHOW PROCEDURE STATUS

```

```

WHERE Db = ' db_4546kyfyj;

```

Db	Name	Type	Language	Definer	Modified	Created	Security_type	Comment	character_set
db_4546kyfyj	ApproveOrRejectLeave	PROCEDURE	SQL	root@%	2026-09-24T05:00:19.000Z	2026-09-24T05:00:19.000Z	DEFINER		utf8mb4
db_4546kyfyj	transfer_employee	PROCEDURE	SQL	user_4546kyfyj@%	2026-09-27T07:29:37.000Z	2026-09-27T07:29:37.000Z	DEFINER		utf8mb4

12. Test the Stored Procedure — Valid Case

First check employee 1:

```

SELECT Emp_ID, Emp_Name, Dept_ID

```

```

FROM Employee

```

```

WHERE Emp_ID = 1;

```

Emp_ID	Emp_Name	Dept_ID
1	Aarav Sharma	1

Employee 1 initially belongs to department 1.

Now transfer the employee to department 2:

```

CALL transfer_employee(1, 2);

```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		16386	0	0

Verify:

```

SELECT Emp_ID, Emp_Name, Dept_ID

```

```

FROM Employee

```

```

WHERE Emp_ID = 1;

```

Emp_ID	Emp_Name	Dept_ID
1	Aarav Sharma	2

13. Edge Case — Employee Does Not Exist

CALL transfer_employee(999, 2);

Error: Employee does not exist

14. Edge Case — Employee Already in Department

Employee 1 is now in department 2.

CALL transfer_employee(1, 2);

Error: Employee is already in this department

15. Test Salary Validation — Negative Salary

UPDATE Employee

SET Salary = -5000

WHERE Emp_ID = 2;

Error: Salary must be greater than zero

16. Test Salary Validation — Salary Below Minimum

UPDATE Employee

SET Salary = 5000

WHERE Emp_ID = 2;

Error: Salary cannot be less than 10000

17. Test Salary Validation — Salary Above Maximum

UPDATE Employee

SET Salary = 1500000

WHERE Emp_ID = 2;

Error: Salary cannot exceed 1000000

18. Test Salary Validation — NULL

UPDATE Employee

SET Salary = NULL

WHERE Emp_ID = 2;

Error: Salary cannot be NULL

19. Test Valid Salary Update

UPDATE Employee

SET Salary = 75000

WHERE Emp_ID = 2;

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	1	0	Rows matched: 1 Changed: 1 Warnings: 0	2	0	1

Check:

SELECT Emp_ID, Emp_Name, Salary

FROM Employee

WHERE Emp_ID = 2;

Emp_ID	Emp_Name	Salary
2	Riya Gupta	75000.00

20. Verify Salary Audit

The previous successful salary update should have generated an audit record.

SELECT *

FROM Salary_Audit

ORDER BY Audit_ID DESC;

Audit_ID	Emp_ID	Old_Salary	New_Salary	Changed_At	Action_Type
1	2	62000.00	75000.00	2026-09-27T07:48:23.000Z	SALARY UPDATE

21. Test INSERT Salary Validation

Try inserting an employee with an invalid salary:

INSERT INTO Employee

(Emp_ID, Emp_Name, Salary, Job_Title, Dept_ID, Project_ID)

VALUES

(31, 'Test Employee', -5000, 'Developer', 1, 101);

Error: Salary must be greater than zero

INSERT INTO Employee

(Emp_ID, Emp_Name, Salary, Job_Title, Dept_ID, Project_ID)

VALUES

(31, 'Test Employee', 45000, 'Developer', 1, 101);

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	1	0		2	0	0

22. Final Verification

Check the employee:

SELECT *

FROM Employee

WHERE Emp_ID = 1;

Emp_ID	Emp_Name	Salary	Job_Title	Dept_ID	Project_ID	Manager_ID
1	Aarav Sharma	65000.00	Developer	2	101	null

Check salary audit:

SELECT *

FROM Salary_Audit

ORDER BY Audit_ID;

Audit_ID	Emp_ID	Old_Salary	New_Salary	Changed_At	Action_Type
1	2	62000.00	75000.00	2026-09-27T07:48:23.000Z	SALARY UPDATE

Check the procedure:

SHOW CREATE PROCEDURE transfer_employee;

Procedure	sql_mode	Create P
transfer_employee	IGNORE_SPACE,ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION	CREATE I

Check triggers:

SHOW TRIGGERS;

Trigger	Event	Table	Statement
validate_salary_insert	INSERT	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG
validate_salary_update	UPDATE	employee	BEGIN IF NEW.Salary IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL'; ELSEIF NEW.Salary <= 0 THEN SIG
salary_audit	UPDATE	employee	BEGIN IF OLD.Salary <> NEW.Salary THEN INSERT INTO Salary_Audit (Emp_ID, Old_Salary, New_Salary, Action_Type) VALUES (NEW.Emp_ID, OL